

Playing with the curves

May 3, 2026

1 Overview

I've studied speaker theory in depth, and have followed Web forums and other online content for years. I've begun to form the opinion that *the Thiele-Small parameters are the wrong way to explain how speakers work. Instead, we should start with the response curves.* Here are my reasons:

1. Your ears are the ultimate judge of course, but the SPL curve is very close in second place for illustrating how a bass speaker is likely to sound, at low frequencies and moderate signal levels.
2. The parameters are abstractions, and involve a level of math and physics that are forbidding to most people.
3. Where do you get the parameters anyway? They're found by measuring curves!
4. Nowadays the designers use the parameters as inputs to their modeling software, which produce curves.
5. You can quickly look at a curve and decide if a design idea stands a chance of working as a bass speaker.

This notebook is the beginning of an attempt to illustrate bass speaker design ideas using curves rather than parameters.

2 My ulterior motive

Where am I headed? I haven't gotten there yet. I want to create an alternative route for learning how speakers work, that *starts with the curves* rather than the parameters. Here, you're watching me thinking my way through this process. In the future, I hope this turns into a useful tutorial.

Cautionary note: This is a work in progress, and may still contain minor bugs. Do not use this program for designing anything that's going to cost money or possibly not work.

3 First, a bit of theory work

Before proceeding, I want to make sure that whatever I'm doing at least doesn't contradict the parameter-based theory that I'm already familiar with. *Mother nature doesn't need two contradictory models of how speakers work, and neither should we.*

But I'm going to take the liberty of simplifying things. This notebook makes a lot of assumptions, even beyond the basic electromechanical theory:

1. I'm still talking about low frequencies and moderate signal levels.
2. I will ignore inductance, since it's mainly applicable to higher frequencies.

3. I will also ignore the overall magnitude of SPL. All curves are pinned down at 0 dB at higher frequencies. There are plenty of equations for overall SPL, or you can get it from the modeling software.

```
[1]: %load_ext autoreload
      %autoreload 2

      import numpy as np
      import matplotlib.pyplot as plt
      import speakerTheorySympy as sps
      import response_curve as rc
```

First, look at the full equation for driver excursion

```
[2]: sps.x_driver
```

```
[2]: 
$$\frac{BLC_{ms}V}{iBL^2C_{ms}\omega - C_{ms}M_{ms}Z_e\omega^2 + iC_{ms}R_{ms}Z_e\omega + Z_e}$$

```

I'm now going to put the Thiele-Small parameters (which I just got done chastizing) back into this equation. Bear with me, I haven't documented this derivation yet.

$$x \propto \frac{1}{f_s(1 + \kappa \frac{V_{as}}{V_{box}}) + \frac{if \cdot f_s}{Q_{ts}} - f^2}$$

where

$$\kappa = \frac{f^2}{f^2 - f_{port}^2} \text{ for ported boxes, and}$$

$\kappa = 1$ for sealed boxes.

What I've done here is to go full circle with the parameters. You can follow this thread in my speaker theory paper, leading to the above result:

1. Start with the T-S parameters, which don't obviously relate to cone motion.
2. Unpacked the Thiele-Small parameters, turning them into electromechanical parameters that a physicist understands.
3. Used the e-m parameters to write an equation of motion for the driver.
4. Solve the equation of motion.
5. Ignore inductance, which makes the equations readable.
6. Put the T-S parameters back in!

Also, getting a bit further ahead of myself, the SPL curve is proportional to:

$$SPL = SPL_0 + 20 \log_{10} \frac{f^2 \kappa}{f_s^2(1 + \kappa \frac{V_{as}}{V_{box}}) + \frac{if \cdot f_s}{Q_{ts}} - f^2}$$

Why is this interesting? SPL as shown here is *the complete equation for the familiar speaker response curve*. Everything about the *sound* of a speaker (at low frequencies and moderate signal levels) is contained in this equation. No magic. The equation is a bit thick, but not all that bad, as it's a function of six parameters:

0. The overall sensitivity SPL_0
1. The resonant frequency of the cone in the box, which is exactly $f_s(1 + \frac{V_{as}}{V_{box}})$ for sealed systems, and slightly lower for ported systems because $\kappa < 1$. We'll see more of κ below.
2. The port tuning frequency f_{port}
3. A small factor hidden in $f_s(1 + \kappa \frac{V_{as}}{V_{box}})$ that I don't have a name for, but slightly modifies SPL below the port tuning frequency. It's the difference between a high-compliance driver in a small box, and a low-compliance driver in a big box, only for ported systems. But huge boxes are not practical. I discovered this factor in the process of writing this article.
4. A total Q factor. Notice that the individual (electrical and mechanical) Q factors don't appear here.
5. While not shown here, inductance is another factor, but has minimal effect around the frequencies where box design is important.

Why bother with the rest of the parameters? Two reasons:

1. The published parameters are *over-complete* meaning that they're not all independent of one another. For instance you don't need Q_{ts} if you've got Q_{es} and Q_{ms} . And the electromechanical parameters (such as BL and M_{ms}) are just a rearrangement of the Thiele-Small parameters.
2. There are more curves, such as impedance, excursion, and port air speed, which are important for the design process, for instance to determine the overall power handling capability of a speaker.

"Speed" of speakers is not a reason. Bassists have been fascinated by "speed" of speakers for as long as I can remember, but time response functions, such as transient and step response, are governed by exactly the same equation as SPL, with the same parameters. I hope to get into this in greater depth, but for now, a decent rule of thumb is that transient response is governed by the ratio of the resonant frequency to the total Q factor. This is precisely true for sealed speakers, and approximately true for ported systems. Since Q doesn't change very much in bass speakers (at most by a factor of 2), that rule boils down to "higher speed means less bass response."

4 Let's look at some curves

4.1 One of my old designs

This is one of the speakers that I've built. My intention was to approximate the response of a small sealed cab in the GK MB150E combo, but with a modern driver and more power handling.

The graph for the GK combo is just a simulation of the GK speaker for the sake of illustration.

I built it about 20 years ago, and my modeling program was a lot more primitive at the time, but it worked. I used this speaker a lot for jazz gigs on both electric and upright bass. You can tell from the curve that there's not a lot of bass response there, but the GK combo didn't have a lot of bass response either.

But this is a useful starting point because it illustrates a lot of points. First, let's look at the SPL calculation:

```
rc.spl_total(f, 53.1, 0.526, 81.1/32, 40)
```

What are these numbers?

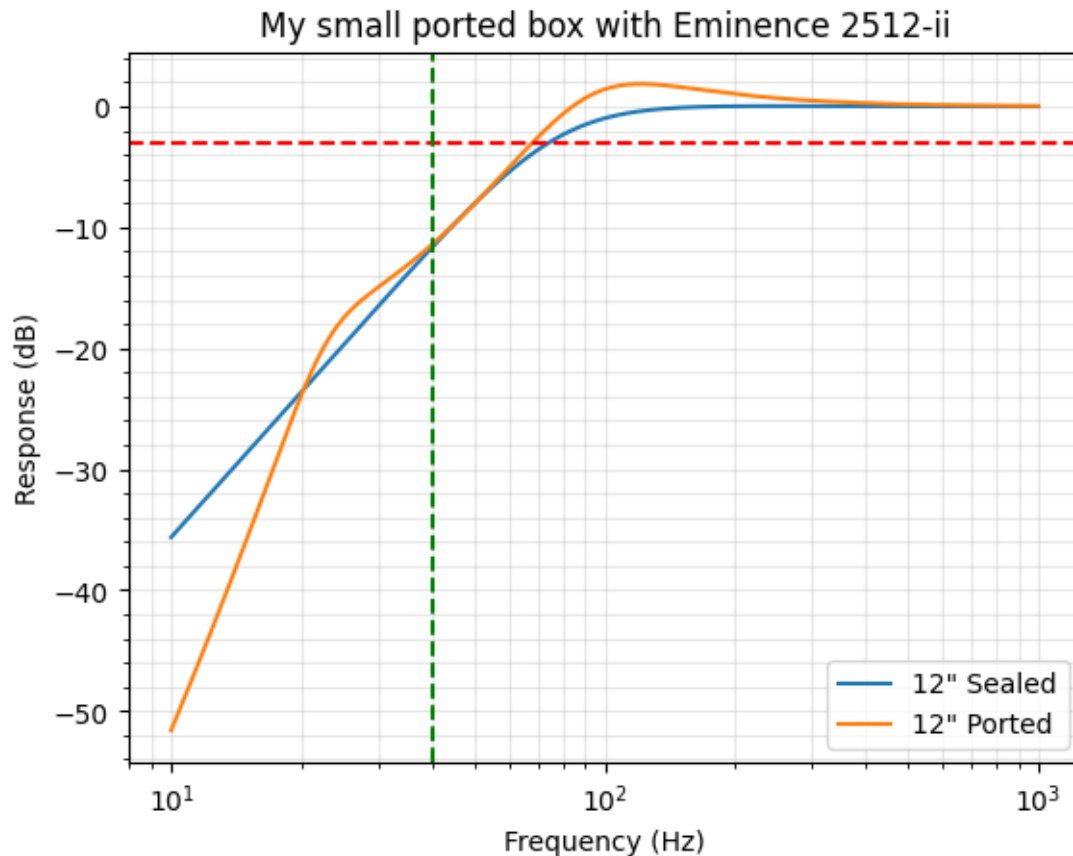
- $F_s = 53.1$, the driver resonant frequency from the Eminence datasheet
- $Q_{ts} = 0.526$, the total Q factor from the datasheet
- $H = 81.1 / 32$, the ratio of the Q_{ts} factor from the datasheet, and the box volume in liters
- F_{port} , the port tuning frequency

I want to point out again that *there's nothing more to it*. This is a complete description of the bass response of this speaker, neglecting overall SPL.

There's a noticeable “hump” in the response curve at the resonant frequency, but it's only 2 dB. It's one of the effects of stuffing this driver into a very small box. A much higher “hump” would lead to “boomy” sound, and would be avoided, but this amount is probably inaudible.

```
[44]: f = np.logspace(1, 3, 1000)
def decorate():
    plt.xlabel('Frequency (Hz)')
    plt.ylabel('Response (dB)')
    plt.gca().axhline(-3, color='red', linestyle='--')
    plt.gca().axvline(40, color='green', linestyle='--')
    plt.minorticks_on()
    plt.grid(which='both', alpha=0.3)
plt.semilogx(f, rc.spl_total(f, 55, 0.526, 1, None), label = '12" Sealed')
plt.semilogx(f, rc.spl_total(f, 53.1, 0.526, 67.44/32, 40), label = '12"
↳Ported')
decorate()
plt.title('My small ported box with Eminence 2512-ii')
plt.legend()
```

```
[44]: <matplotlib.legend.Legend at 0x7ada65a14410>
```

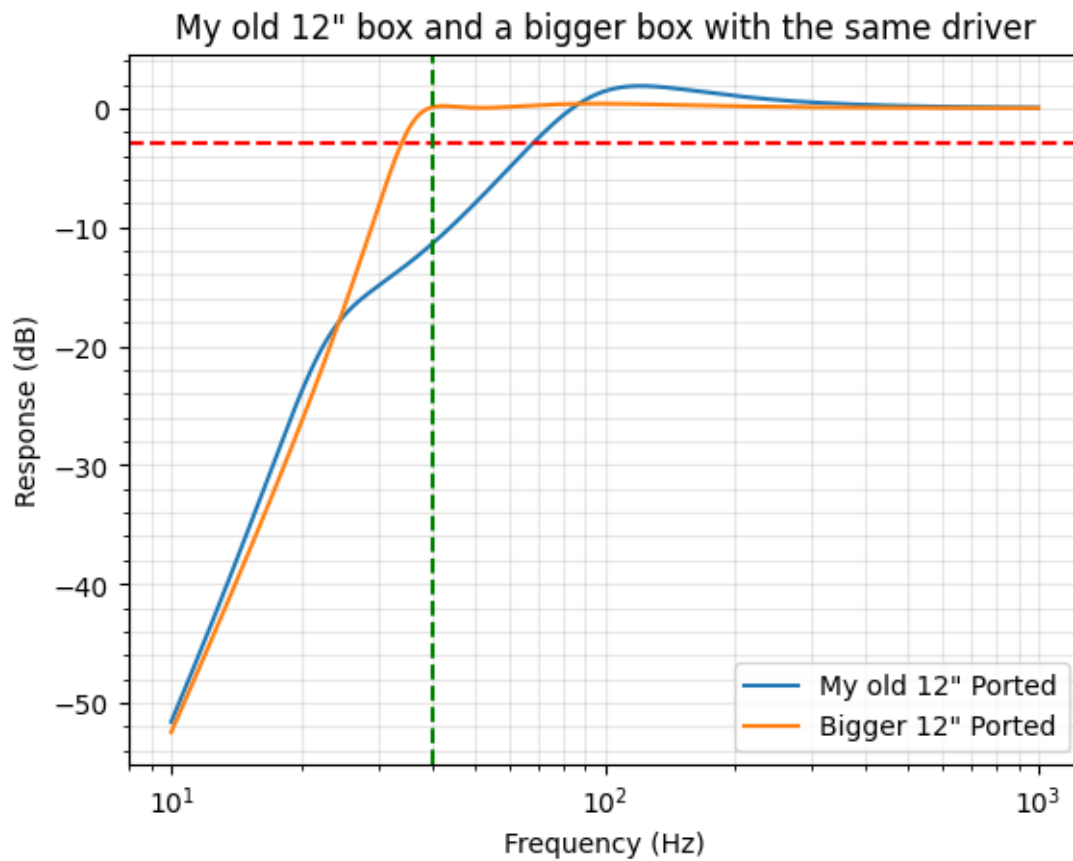


4.2 A bigger box for this driver?

You bet. Putting this driver in a much bigger box extends its low cutoff frequency down to about 35 Hz, and there's no in-between curve. But it's a huge box. Also, it will be limited in its power handling capability. But it shows that a single driver can produce two different kinds of sound quality depending on box and port design. *This would be quite an impressive woofer for a hi-fi system if you had space for it.*

```
[45]: plt.semilogx(f, rc.spl_total(f, 53.1, 0.526, 67.44/32, 40), label = 'My old 12" Ported')
plt.semilogx(f, rc.spl_total(f, 53.1, 0.526, 67.44/120, 40), label = 'Bigger 12" Ported')
decorate()
plt.title('My old 12" box and a bigger box with the same driver')
plt.legend()
```

```
[45]: <matplotlib.legend.Legend at 0x7ada64f14fb0>
```



```
[26]: f = np.logspace(1, 3, 1000)
f_s = 53.1
H = 67.44 / 35.17 #  $V_{as} / V_b$ 
Q_ts = 0.526
f_port = 40

f_s = 52
H = 81.1 / 32 #  $V_{as} / V_b$ 
Q_ts = 0.318
f_port = 63.91

print('cone/box resonant frequency (Hz)', f_s*np.sqrt(1 + H))
```

cone/box resonant frequency (Hz) 97.75965425470775

5 Some basic graphs

5.1 The port tuning function

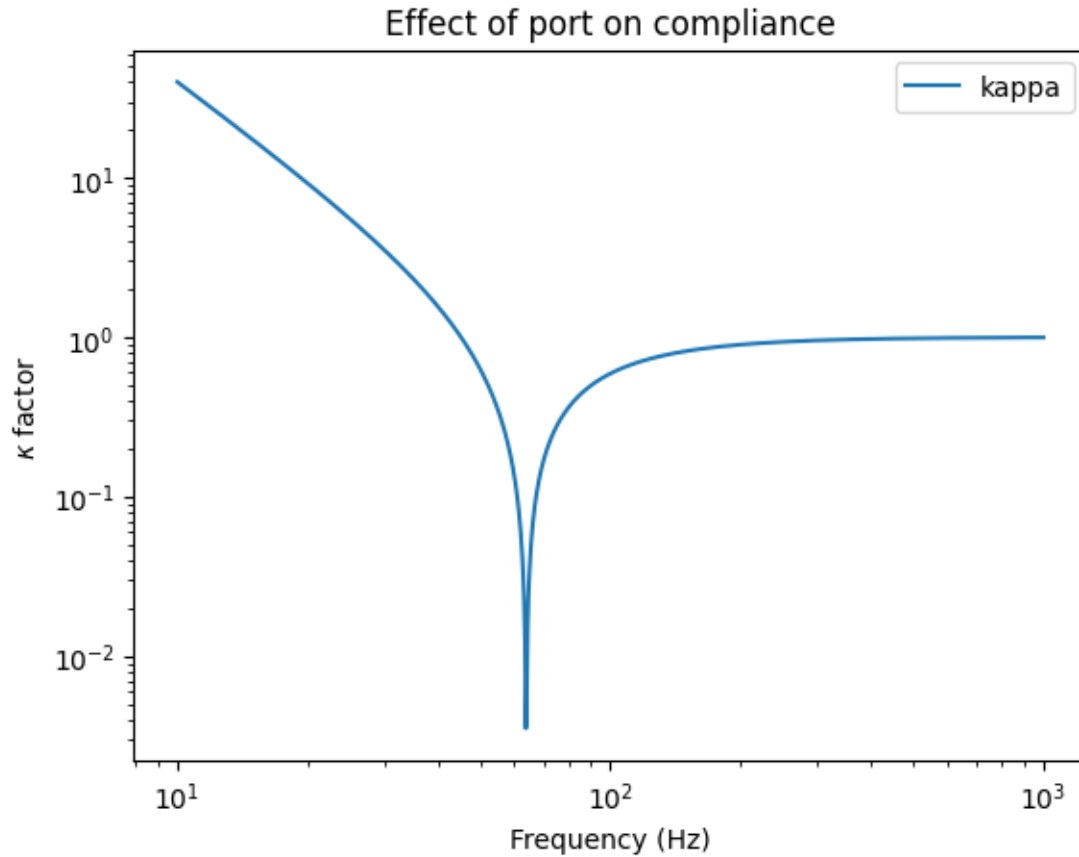
The function $\kappa(f)$ governs the relationship between the motion of the cone and the air mass in the port. It also affects the compliance of the box as “seen” by the cone. It’s kind of an awkward term, but compliance is the inverse of stiffness.

Above the port tuning frequency, the compliance of the box returns to the same value as a sealed box of the same size. Below, the compliance reverts to the compliance of the cone. This is what’s meant when it’s said that the cone is *unsupported* below the tuning frequency.

The stiffness of the box / port system at the tuning frequency is why cone motion drops to a minimum, as we’ll see below.

```
[18]: def kappa(f, f_port):  
    '''  
    Correction factor for port tuning  
    '''  
    if f_port is None:  
        return 1  
    else:  
        return f**2/(f**2 - f_port**2)
```

```
[52]: # plt.semilogx(f, 20*np.log10(np.abs(1/kappa(f, f_port))), label = 'kappa')  
plt.loglog(f, np.abs(1/kappa(f, f_port)), label = 'kappa')  
plt.title('Effect of port on compliance')  
plt.xlabel('Frequency (Hz)')  
plt.ylabel(r'$\kappa$ factor')  
plt.legend()  
plt.show()
```



5.2 Cone excursion

It's worth looking at cone excursion by itself. This is a graph that the designers look at, to help estimate the power handling capability of a speaker. It also illustrates a benefit of ported design.

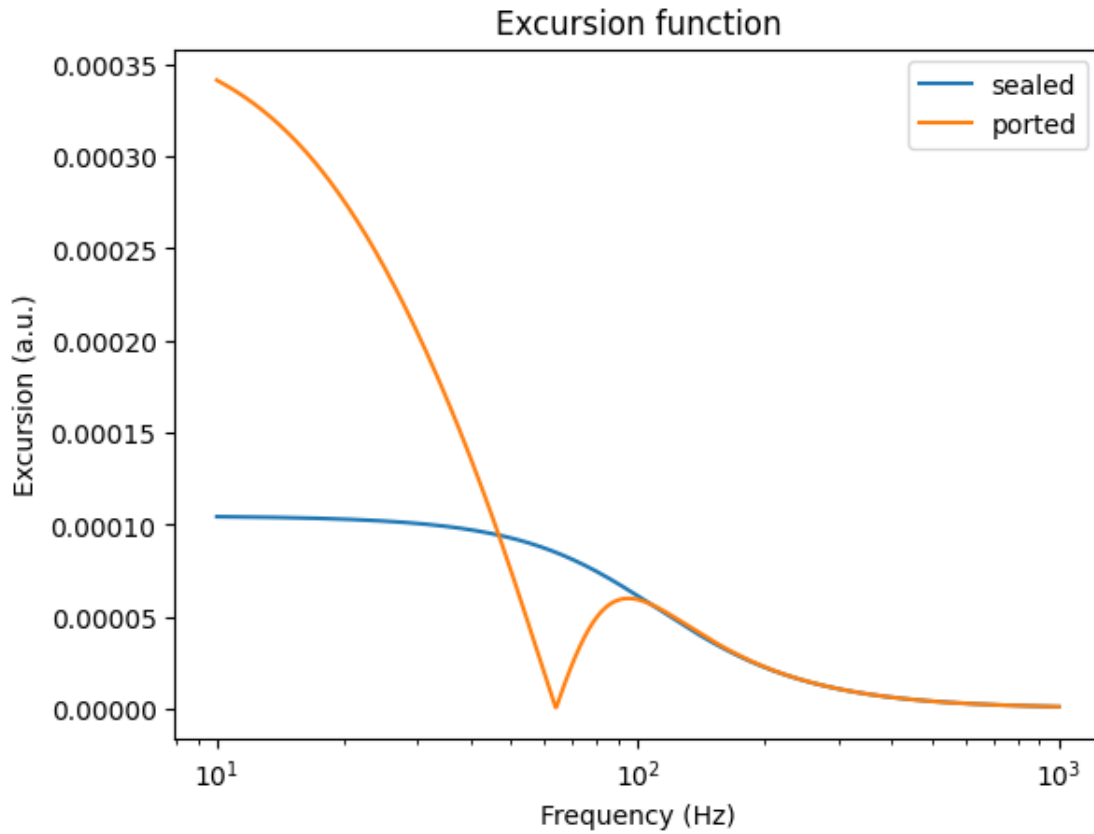
In the sealed box, excursion rises to a high level at low frequencies, which can lead to distortion or even driver damage. The port takes some of the load off of the cone, replacing it with motion of the air mass in the port itself. With careful design, a ported system can handle more power than a sealed box.

```
[20]: def x(f, f_s, Q_ts, H, f_port):
        """
        Excursion function
        """
        return 1/(f_s**2*(1 + kappa(f, f_port)*H) + 1j*f*f_s/Q_ts - f**2)
```

```
[53]: plt.semilogx(f, np.abs(x(f, f_s, Q_ts, H, None)), label = 'sealed')
plt.semilogx(f, np.abs(x(f, f_s, Q_ts, H, f_port)), label = 'ported')
plt.legend()
plt.title('Excursion function')
```



```
plt.xlabel('Frequency (Hz)')
plt.ylabel('Excursion (a.u.)')
plt.show()
```

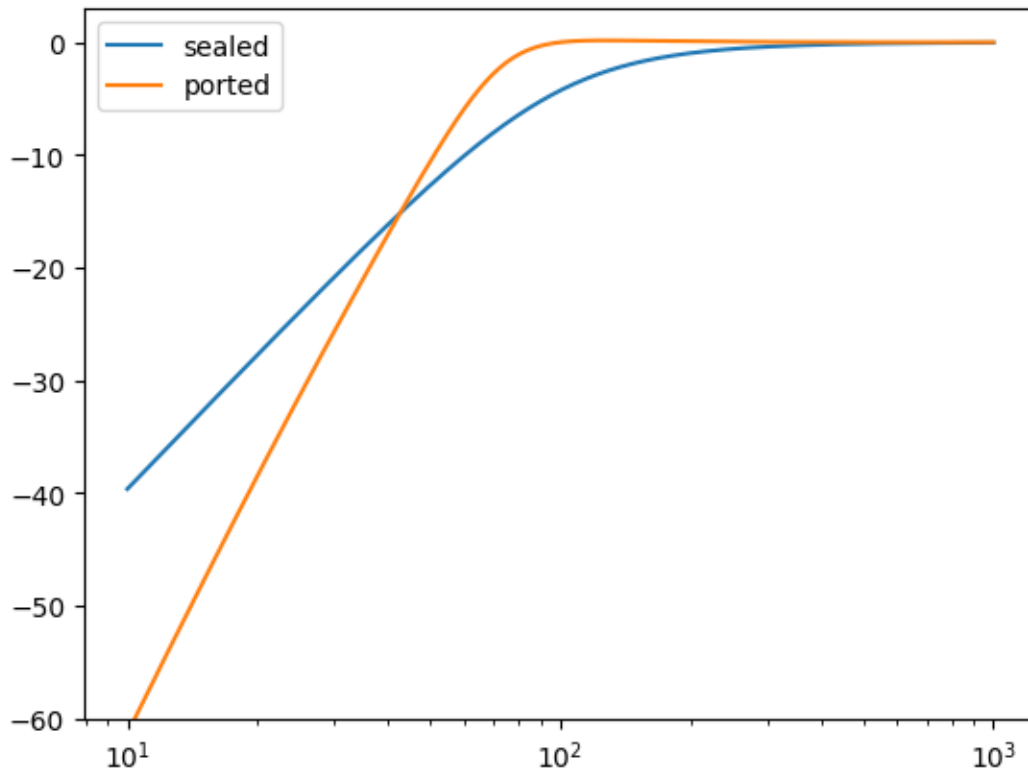


5.3 The SPL curve

For understanding speakers, I think the SPL curve is the closest to a representation of the sound of a speaker.

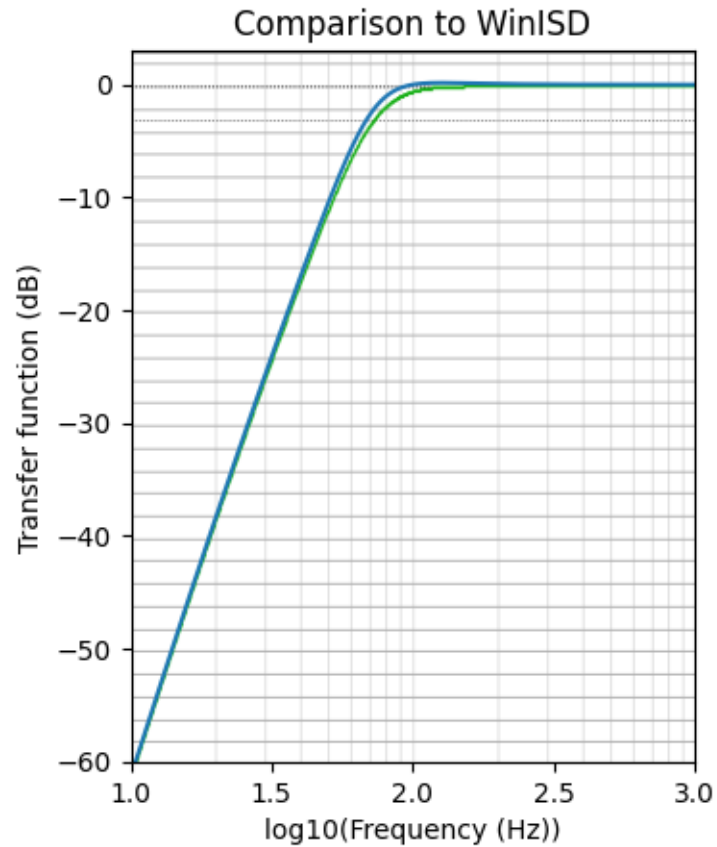
```
[27]: def spl_total(f, f_s, Q_ts, H, f_port):
        '''
        Total sound pressure level function
        '''
        return 20*np.log10(np.abs(f**2*kappa(f, f_port)*x(f, f_s, Q_ts, H, f_port)))
plt.figure()
plt.semilogx(f, spl_total(f, f_s, Q_ts, H, None), label = 'sealed')
plt.semilogx(f, spl_total(f, f_s, Q_ts, H, f_port), label = 'ported')
plt.ylim(-60, 3)
plt.legend()
```

[27]: <matplotlib.legend.Legend at 0x764b151acfb0>



```
[39]: img = plt.imread("img/spl_winisd_2.png")
plt.imshow(img, extent=[1, 3, -60, 3], aspect = 0.04)
plt.plot(np.log10(f), spl_total(f, f_s, Q_ts, H, f_port), label = 'ported')
plt.ylim(-60, 3)
plt.title('Comparison to WinISD')
plt.xlabel('log10(Frequency (Hz))')
plt.ylabel('Transfer function (dB)')
```

[39]: Text(0, 0.5, 'Transfer function (dB)')



```
[ ]: # Code for converting this document to HTML
if __name__ == '__main__':
    if input('Save this notebook before proceeding. OK? (y/n)') == 'y':
        !python3 -m jupyter nbconvert --to pdf ./"Playing with the curves.ipynb"
```

```
[ ]:
```